Design Patterns in a Rails Booking Application

Source evidence, requirement evolution and four-language comparison

Team A7
Kavya Penikalapati E. Abhijith Krishna
Guduru Akshaya Giri Kurt Sony Rebello

23CSE455 – B.Tech Artificial Intelligence & Data Science

6 September 2027

A fixed baseline makes the findings reproducible

Application: `andrewdsilva/rails-room-booking`

- Ruby 2.7.5, Rails 7.0.4 dependency and PostgreSQL configuration.
- Commit: `5e15551ee646871b4bed32c982fa17da319d3ee3`.
- Rails MVC and Active Record organize User, Room, Booking and Setting.
- Devise, Pundit, Payify/Stripe and webhooks are integration boundaries.

Method: establish participant roles, calls and purpose in source before assigning a GoF pattern.

One baseline collaboration supports Template Method

Stable CRUD skeleton

1. Authorize and construct the resource.
2. Validate and save it.
3. Redirect on success, or render errors.

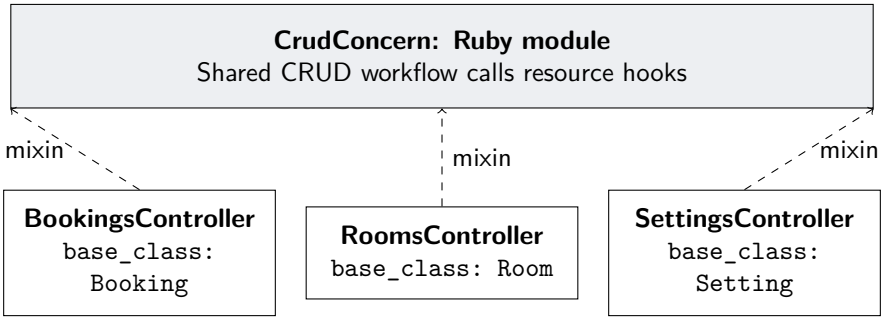
`CrudConcern` is a Ruby module mixed into three controllers.

Resource hooks vary

- `base_class`, `object_class`
- `model_params`, `strong_params`
- Settings overrides lookup and redirect.

Required hooks raise `NotImplementedError` in the shared module.

Module inclusion expresses the Rails collaboration



`include ::CrudConcern` mixes in the skeleton.
Controllers supply hooks and inherit ApplicationController.

Framework vocabulary alone does not prove a pattern

Candidate	Evidence boundary
Observer	Callbacks do not establish application-owned observer objects.
Strategy	Pundit policies do not establish the claimed interchangeable algorithm context.
Decorator	A concern adds methods through module inclusion; runtime object wrapping is absent.
Adapter / Facade	External Payify internals were not treated as application source.

Four requirements create three distinct design pressures

Requirement	Responsibility	Implementation
Breakfast / airport transfer	Optional booking services	AdditionalService subclasses and BookingService records
Room upgrade	Reservation modification	RoomUpgrade checks availability and recalculates price
Cancellation rules	Variable algorithms	CancellationService delegates to policy objects

Template Method remains; cancellation introduces Strategy. The other changes are domain modeling and responsibility extraction.

Cancellation delegates refund rules to a Strategy

Context: CancellationService

1. Ask the policy whether cancellation is permitted.
2. Ask it to calculate the refund.
3. Persist the cancellation time and calculated refund; return the result.

Interchangeable policies

- FlexibleCancellationPolicy
- PartialRefundCancellationPolicy
- NoRefundCancellationPolicy

A new rule changes the policy family without adding branches to the orchestration.

The two pattern ideas run in four languages

Language	Contract mechanism
Java	Abstract workflow class and policy interface
Python	Abstract base classes and dynamic dispatch
JavaScript	Class methods and runtime polymorphism
C++	Pure virtual methods and explicit ownership

Eight isolated snippets passed behavioral smoke checks. Strategy refunds for amount 430: full 430, partial 215, no refund 0.

Process timing describes this environment

Language	Template Method (ms)	Strategy (ms)
Java	39.494	50.372
Python	672.008	595.258
JavaScript	21.401	21.762
C++	1.097	1.223

One warmup, then five measured processes; median reported. Startup is included. Tiny examples and different runtimes prevent universal language-speed conclusions.

Evidence determines the strength of each conclusion

- Source traces establish Template Method and the introduced Strategy.
- Part 3 outputs establish behavior of the eight isolated examples.
- Rails: 36 examples, zero failures; all 12 migrations up; real HTTP startup returns the expected redirect.
- Historical Prompt 02/03 wording and some model/date metadata were not retained; gaps are disclosed.

Finding: use a pattern when the collaboration and design pressure justify it. A smaller evidence-backed inventory is easier to defend.